
Points-to Analysis

Lecture 8 — Static Analysis by Abstract Interpretation

David Pichardie

Fall 2026

A minimal language with pointers

A flow-sensitive points-to interpreter

Flow-insensitive points-to analysis

§ 1

A minimal language with pointers

Why points-to information?

Consider

```
p := &x;  
q := p;  
*q := &y
```

The last command modifies `x`, although `x` does not occur syntactically on its left-hand side.

Question answered by points-to analysis

At each program point, which memory cells may each pointer expression designate?

This information supports alias analysis, null-dereference checks, indirect-call resolution, and many later optimizations.

Pointer fragment of the language

We isolate a pointer-only fragment of the imperative language:

$$\begin{aligned} e &::= \text{null} \mid \&x \mid *e, \\ c &::= \text{skip} \mid *e_1 := e_2 \mid x := \text{alloc}^a() \\ &\quad \mid c_1; c_2 \mid \text{if } * \text{ then } c_1 \text{ else } c_2 \mid \text{while } * \text{ do } c. \end{aligned}$$

- The label a uniquely identifies an allocation site in the source code.
- Each time the guard $*$ is evaluated, it nondeterministically returns either `tt` or `ff`.
- A bare variable is not an expression. Reading the content of x is written $*\&x$, and an ordinary write is written $*\&x := e$. Allocation retains the dedicated form $x := \text{alloc}^a()$. For readability, we sometimes write x instead of $*\&x$.
- We omit integers and pointer arithmetic: values are only pointers or `null`.

Example — appending to a linked list

```
p := &head;  
while * do {  
    p := *p;  
};  
q := alloca();  
*q := null;  
*p := q
```

- Each node is one heap cell containing the address of the next node; null marks the end.
- The variable *p* contains the address of the current link cell: either *head* or a node.
- The guard *** abstracts the concrete test against null and may stop at any iteration.

Concrete types

$$@x \in \text{VarLoc} \triangleq \text{Var},$$

$$\ell \in \text{Loc} \triangleq \text{VarLoc} \uplus \text{HeapLoc}.$$

$$v \in \text{Val} \triangleq \text{Loc} \uplus \{\varepsilon\},$$

$$\mu \in \text{Mem} \triangleq \text{Loc} \rightarrow \text{Val}.$$

A *location* is the concrete identity of one memory cell. It is either:

- a program variable, giving its permanent addressable cell;
- an opaque heap identifier $h \in \text{HeapLoc}$ created dynamically by allocation.

A value is either a location ℓ or ε , which represents the null pointer. The notation $@x$ distinguishes a variable cell from the value currently stored in that cell.

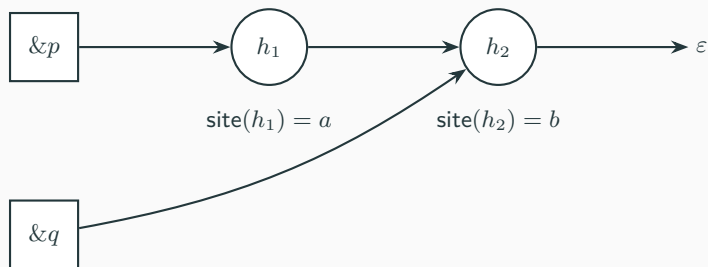
Each opaque heap identifier carries an immutable allocation-site tag:

$$\text{site} : \text{HeapLoc} \longrightarrow \text{AllocSite}.$$

The tag is the a appearing in $\text{alloc}^a()$.

Concrete memory graph

Consider the concrete memory obtained after

$$p := \text{alloc}^a(); \quad q := \text{alloc}^b(); \quad *p := q.$$


An edge $\ell \longrightarrow \ell'$ means $\mu(\ell) = \ell'$.

Exercise — draw a concrete memory

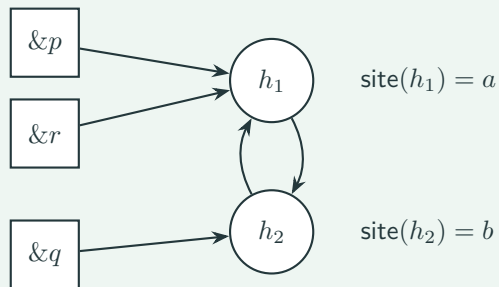
Initially, p , q , and r contain null. Execute

```
 $p := \text{alloc}^a();$   
 $q := \text{alloc}^b();$   
 $r := p;$   
 $*p := q;$   
 $*q := p.$ 
```

Draw the final concrete memory graph and indicate the allocation-site tag of each heap location.

Exercise — draw a concrete memory

Solution



Big-step evaluation of pointer expressions

The judgement $\mu \vdash e \Downarrow v$ evaluates e in memory μ .

$$\frac{}{\mu \vdash \text{null} \Downarrow \varepsilon} \text{ NULL}$$

$$\frac{}{\mu \vdash \&x \Downarrow @x} \text{ ADDRESS}$$

$$\frac{\mu \vdash e \Downarrow \ell \quad \ell \in \text{dom}(\mu) \quad \mu(\ell) = v}{\mu \vdash *e \Downarrow v} \text{ Deref}$$

Big-step semantics — commands

Write $\langle c, \mu \rangle \Downarrow \mu'$ when c terminates from μ in μ' .

$$\begin{array}{c} \frac{}{\langle \text{skip}, \mu \rangle \Downarrow \mu} \text{ SKIP} \qquad \frac{h \notin \text{dom}(\mu) \quad \text{site}(h) = a}{\langle x := \text{alloc}^a(), \mu \rangle \Downarrow \mu[@x \mapsto h, h \mapsto \varepsilon]} \text{ ALLOC} \\[1em] \frac{\mu \vdash e_1 \Downarrow \ell \quad \ell \in \text{dom}(\mu) \quad \mu \vdash e_2 \Downarrow v}{\langle *e_1 := e_2, \mu \rangle \Downarrow \mu[\ell \mapsto v]} \text{ STORE} \end{array}$$

Allocation chooses any fresh h tagged with a . Dereferencing ε has no rule: the execution is stuck.

Big-step semantics — control flow

$$\frac{\langle c_1, \mu \rangle \Downarrow \mu' \quad \langle c_2, \mu' \rangle \Downarrow \mu''}{\langle c_1; c_2, \mu \rangle \Downarrow \mu''} \text{ SEQ}$$

$$\frac{\langle c_1, \mu \rangle \Downarrow \mu'}{\langle \text{if } * \text{ then } c_1 \text{ else } c_2, \mu \rangle \Downarrow \mu'} \text{ IF-LEFT}$$

$$\frac{\langle c_2, \mu \rangle \Downarrow \mu'}{\langle \text{if } * \text{ then } c_1 \text{ else } c_2, \mu \rangle \Downarrow \mu'} \text{ IF-RIGHT}$$

$$\frac{}{\langle \text{while } * \text{ do } c, \mu \rangle \Downarrow \mu} \text{ WHILE-STOP}$$

$$\frac{\langle c, \mu \rangle \Downarrow \mu' \quad \langle \text{while } * \text{ do } c, \mu' \rangle \Downarrow \mu''}{\langle \text{while } * \text{ do } c, \mu \rangle \Downarrow \mu''} \text{ WHILE-ITERATE}$$

The guard `*` makes the semantics nondeterministic: either branch may be selected, and a loop may execute any finite number of iterations.

Exercise — execute a pointer program

Initially,

$$\mu_0 = [\text{@}x \mapsto \varepsilon, \text{@}y \mapsto \varepsilon, \text{@}p \mapsto \varepsilon, \text{@}q \mapsto \varepsilon].$$

Compute the final memory:

`p := &x; q := p; *q := &y; p := *q.`

Which cells are modified by each command?

Exercise — execute a pointer program

Solution

	$@x$	$@y$	$@p$	$@q$
μ_0	ε	ε	ε	ε
μ_1	ε	ε	$@x$	ε
μ_2	ε	ε	$@x$	$@x$
μ_3	$@y$	ε	$@x$	$@x$
μ_4	$@y$	ε	$@y$	$@x$

The indirect store modifies x because q contains the address x .

§2

A flow-sensitive points-to interpreter

One abstract heap cell per allocation site

A concrete execution may allocate arbitrarily many locations. Group them by allocation site to obtain a finite abstraction:

Concrete and abstract types

Concrete

$$\ell \in \text{Loc} \triangleq \text{VarLoc} \uplus \text{HeapLoc},$$

$$v \in \text{Val} \triangleq \text{Loc} \uplus \{\varepsilon\},$$

$$\mu \in \text{Mem} \triangleq \text{Loc} \rightarrow \text{Val}.$$

Abstract

$$\hat{\ell} \in \text{ALoc} \triangleq \text{VarLoc} \uplus \text{AllocSite},$$

$$\hat{v} \in \text{AVal} \triangleq \text{ALoc} \uplus \{\varepsilon\},$$

$$\hat{\mu} \in \text{AMem} \triangleq \text{ALoc} \rightarrow \mathcal{P}(\text{AVal}).$$

The abstraction observes only the immutable tag of a heap location:

$$\beta_{\text{Loc}}(@x) = @x, \quad \beta_{\text{Loc}}(h) = \text{site}(h),$$

$$\beta_{\text{Val}}(\varepsilon) = \varepsilon, \quad \beta_{\text{Val}}(\ell) = \beta_{\text{Loc}}(\ell).$$

The maps β_{Loc} and β_{Val} are called *representation functions*. This approach applies whenever $\alpha(S) = \{\beta(s) \mid s \in S\}$.

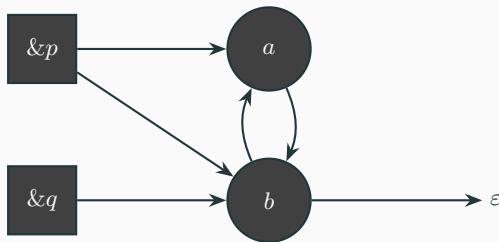
Abstract states are points-to graphs

An edge $\hat{\ell} \longrightarrow \hat{\ell}'$ means:

$$\hat{\ell}' \in \hat{\mu}(\hat{\ell}) \iff \hat{\ell} \text{ may point to } \hat{\ell}'.$$

An edge from $\hat{\ell}$ to ε denotes $\varepsilon \in \hat{\mu}(\hat{\ell})$.

$$\hat{\mu} = [\text{@}p \mapsto \{a, b\}, \text{@}q \mapsto \{b\}, a \mapsto \{b\}, b \mapsto \{a, \varepsilon\}].$$



$$\gamma : \text{AMem} \longrightarrow \mathcal{P}(\text{Mem}).$$

Meaning of a points-to graph

$$\gamma(\widehat{\mu}) \triangleq \{\mu \mid \forall \ell \in \text{dom}(\mu), \beta_{\text{Val}}(\mu(\ell)) \in \widehat{\mu}(\beta_{\text{Loc}}(\ell))\}.$$

Every concrete edge must occur abstractly; extra edges express *may-points-to* information.

The concrete memory graph is *deterministic*: every cell has exactly one outgoing edge, to a location or to ε . The abstract graph need not be: several outgoing edges represent several possible values.

The precision order and join are pointwise:

$$\hat{\mu}_1 \sqsubseteq \hat{\mu}_2 \iff \forall \hat{\ell}, \hat{\mu}_1(\hat{\ell}) \subseteq \hat{\mu}_2(\hat{\ell}), \quad (\hat{\mu}_1 \sqcup \hat{\mu}_2)(\hat{\ell}) = \hat{\mu}_1(\hat{\ell}) \cup \hat{\mu}_2(\hat{\ell}).$$

$$\perp(\hat{\ell}) = \emptyset, \quad \top(\hat{\ell}) = \text{AVal}.$$

- More edges mean more possible concrete memories, hence less precision.
- The domain is finite for a fixed program: variables and allocation sites are finite.
- Therefore ascending Kleene iteration terminates; no widening is needed.

Exercise — the best abstract graph

Consider the following concrete memories:

$$\begin{array}{lll} \mu_1 & = & [\textcircled{p} \mapsto h_1, h_1 \mapsto \varepsilon], \quad \text{site}(h_1) = a, \\ \mu_2 & = & [\textcircled{p} \mapsto h_2, h_2 \mapsto h_2], \quad \text{site}(h_2) = a, \\ \mu_3 & = & [\textcircled{p} \mapsto h_3, h_3 \mapsto \varepsilon], \quad \text{site}(h_3) = b. \end{array}$$

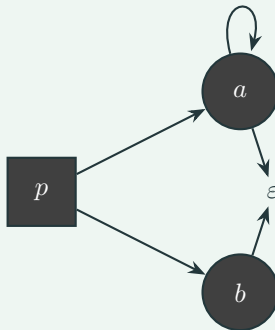
Give the best abstraction of $\{\mu_1, \mu_2, \mu_3\}$:

1. as a map $\widehat{\mu}$;
2. as a points-to graph, using filled vertices.

Exercise — the best abstract graph

Solution

$$\hat{\mu} = [\textcircled{p} \mapsto \{a, b\}, a \mapsto \{\varepsilon, a\}, b \mapsto \{\varepsilon\}] \quad (\text{empty elsewhere}).$$



Pointer expressions evaluate to sets of possible abstract values:

$$\begin{aligned}\text{eval}^\#(\text{null}, \hat{\mu}) &= \{\varepsilon\}, \\ \text{eval}^\#(\&x, \hat{\mu}) &= \{ @x \}, \\ \text{eval}^\#(*e, \hat{\mu}) &= \bigcup_{\hat{\ell} \in \text{eval}^\#(e, \hat{\mu})} \hat{\mu}(\hat{\ell}).\end{aligned}$$

Expression soundness

If $\mu \in \gamma(\hat{\mu})$ and $\mu \vdash e \Downarrow v$, then

$$\beta_{\text{Val}}(v) \in \text{eval}^\#(e, \hat{\mu}).$$

The proof is by structural induction on e .

The store $*\&x := e$ writes one known stack cell:

$$\llbracket *\&x := e \rrbracket^\#(\hat{\mu}) = \hat{\mu}[\@x \mapsto \text{eval}^\#(e, \hat{\mu})].$$

This is a *strong update*: the old outgoing edges of x are removed.

Important asymmetry. The restriction of β_{Loc} to VarLoc is injective; in particular, $\beta_{\text{Loc}}^{-1}(\@x) = \{\@x\}$. This is what makes the strong update sound. It is not injective on HeapLoc : distinct locations allocated at the same site have the same representation.

Soundness of the strong update

Assume $\mu \in \gamma(\widehat{\mu})$ and $\mu \vdash e \Downarrow v$, and define

$$\mu' = \mu[@x \mapsto v], \quad \widehat{\mu}' = \widehat{\mu}[@x \mapsto \text{eval}^\sharp(e, \widehat{\mu})].$$

We prove $\mu' \in \gamma(\widehat{\mu}')$. By expression soundness,

$$\beta_{\text{Val}}(v) \in \text{eval}^\sharp(e, \widehat{\mu}).$$

Let $\ell \in \text{dom}(\mu')$. We must show

$$\beta_{\text{Val}}(\mu'(\ell)) \in \widehat{\mu}'(\beta_{\text{Loc}}(\ell)).$$

1. If $\ell = @x$, then $\beta_{\text{Loc}}(\ell) = @x$ and

$$\beta_{\text{Val}}(\mu'(\ell)) = \beta_{\text{Val}}(v) \in \text{eval}^\sharp(e, \widehat{\mu}) = \widehat{\mu}'(@x).$$

2. If $\ell \neq @x$, then $\beta_{\text{Loc}}(\ell) \neq @x$ because $\beta_{\text{Loc}}^{-1}(@x) = \{@x\}$. Thus this abstract cell is unchanged, and

$$\beta_{\text{Val}}(\mu'(\ell)) = \beta_{\text{Val}}(\mu(\ell)) \in \widehat{\mu}(\beta_{\text{Loc}}(\ell)) = \widehat{\mu}'(\beta_{\text{Loc}}(\ell)),$$

using $\mu \in \gamma(\widehat{\mu})$.

The condition defining $\gamma(\widehat{\mu}')$ therefore holds for every $\ell \in \text{dom}(\mu')$, so $\mu' \in \gamma(\widehat{\mu}')$.

The minimal sound transfer weakly updates every possible target:

$$e_1 \notin \text{Var} \implies \llbracket *e_1 := e_2 \rrbracket^\#(\hat{\mu})(\hat{\ell}) = \begin{cases} \hat{\mu}(\hat{\ell}) \cup \text{eval}^\#(e_2, \hat{\mu}) & \hat{\ell} \in \text{eval}^\#(e_1, \hat{\mu}), \\ \hat{\mu}(\hat{\ell}) & \hat{\ell} \notin \text{eval}^\#(e_1, \hat{\mu}). \end{cases}$$

Soundness of the indirect store

Assume $\mu \in \gamma(\widehat{\mu})$, $\mu \vdash e_1 \Downarrow \ell_0$, and $\mu \vdash e_2 \Downarrow v$. Define

$$\mu' = \mu[\ell_0 \mapsto v], \quad \widehat{\mu}' = \llbracket *e_1 := e_2 \rrbracket^\#(\widehat{\mu}).$$

Expression soundness gives $\beta_{\text{Loc}}(\ell_0) \in \text{eval}^\#(e_1, \widehat{\mu})$ and $\beta_{\text{Val}}(v) \in \text{eval}^\#(e_2, \widehat{\mu})$.

For every $\ell \in \text{dom}(\mu')$:

1. If $\ell = \ell_0$, then

$$\beta_{\text{Val}}(\mu'(\ell)) = \beta_{\text{Val}}(v) \in \text{eval}^\#(e_2, \widehat{\mu}) \subseteq \widehat{\mu}'(\beta_{\text{Loc}}(\ell_0)).$$

2. If $\ell \neq \ell_0$, then $\mu'(\ell) = \mu(\ell)$. Moreover, the weak update only adds values, hence

$$\beta_{\text{Val}}(\mu'(\ell)) = \beta_{\text{Val}}(\mu(\ell)) \in \widehat{\mu}(\beta_{\text{Loc}}(\ell)) \subseteq \widehat{\mu}'(\beta_{\text{Loc}}(\ell)).$$

Thus $\mu' \in \gamma(\widehat{\mu}')$.

For $x := \text{alloc}^a()$, every fresh concrete location is represented by the allocation site a :

$$\llbracket x := \text{alloc}^a() \rrbracket^\#(\hat{\mu}) = \hat{\mu}[@x \mapsto \{a\}, a \mapsto \hat{\mu}(a) \cup \{\varepsilon\}].$$

- Updating $@x$ is strong: it represents one concrete cell.
- Updating a is weak: it may summarize objects created by previous executions of the same site.

The complete abstract interpreter

The command interpreter has exactly the structure used in Lectures 4–6:

$$\begin{aligned}\llbracket c_1; c_2 \rrbracket^\sharp &= \llbracket c_2 \rrbracket^\sharp \circ \llbracket c_1 \rrbracket^\sharp, \\ \llbracket \text{if } * \text{ then } c_1 \text{ else } c_2 \rrbracket^\sharp(\hat{\mu}) &= \llbracket c_1 \rrbracket^\sharp(\hat{\mu}) \sqcup \llbracket c_2 \rrbracket^\sharp(\hat{\mu}), \\ \llbracket \text{while } * \text{ do } c \rrbracket^\sharp(\hat{\mu}) &= \text{lfp}(\lambda X. \hat{\mu} \sqcup \llbracket c \rrbracket^\sharp(X)).\end{aligned}$$

Termination

All primitive transformers are monotone, and the points-to lattice is finite. Kleene iteration therefore reaches the abstract least fixed point.

Let $\llbracket c \rrbracket$ be the collecting semantics induced by the big-step rules.

Sound points-to analysis

For every command c and abstract state $\hat{\mu}$,

$$\boxed{\llbracket c \rrbracket(\gamma(\hat{\mu})) \subseteq \gamma(\llbracket c \rrbracket^\#(\hat{\mu}))}.$$

The proof is similar to the soundness proofs for signs and intervals.

Exercise — flow-sensitive propagation

Assume initially

$$\mu_0 = [\textcircled{x} \mapsto \textcircled{u}, \textcircled{y} \mapsto \varepsilon, \textcircled{p} \mapsto \varepsilon, \textcircled{q} \mapsto \varepsilon, \textcircled{r} \mapsto \varepsilon].$$

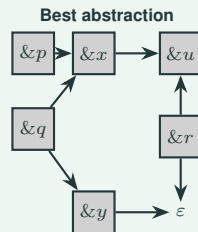
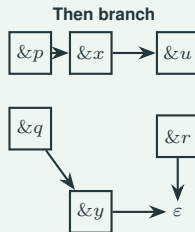
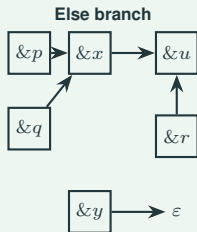
Execute

1. `p := &x;`
2. `q := p;`
3. `if * then q := &y else skip;`
4. `r := *q.`

Draw the two possible final concrete memories and the best abstract memory representing both.
Also give the abstract memory after each numbered line.

Exercise — flow-sensitive propagation

Solution



after	$\hat{\mu}(@p)$	$\hat{\mu}(@q)$	$\hat{\mu}(@r)$
1	$\{@x\}$	$\{\varepsilon\}$	$\{\varepsilon\}$
2	$\{@x\}$	$\{@x\}$	$\{\varepsilon\}$
3	$\{@x\}$	$\{@x, @y\}$	$\{\varepsilon\}$
4	$\{@x\}$	$\{@x, @y\}$	$\{@u, \varepsilon\}$

$$\hat{\mu}(@x) = \{@u\},$$

$$\hat{\mu}(@y) = \{\varepsilon\}.$$

Exercise — a spurious concrete memory

Initially, every variable contains ε . Analyze

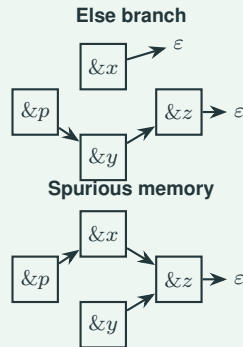
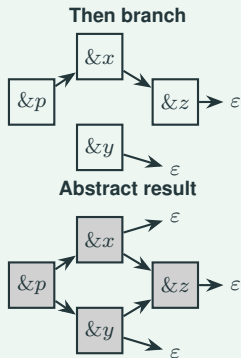
```
if * then p := &x else p := &y;    *p := &z.
```

At the end:

1. draw the two possible concrete memories;
2. draw the abstract memory computed by the analysis;
3. draw a concrete memory represented by this abstraction that no execution can produce;

Exercise — a spurious concrete memory

Solution



The store modifies only one concrete cell. Since $\hat{\mu}(@p) = \{@x, @y\}$, the analysis weakly updates both $@x$ and $@y$; the last graph, where both are modified, is therefore represented but unreachable.

§3

Flow-insensitive points-to analysis

Do we need one graph at every program point?

The interpreter built so far is *flow-sensitive*:

each program point has its own abstract state.

This preserves command order and enables strong updates, but requires propagating graphs through the control-flow graph and solving loop fixed points.

A *flow-insensitive* analysis instead computes one graph $\hat{\mu}$ valid everywhere:

all pointer assignments contribute constraints to one solution.

It ignores control flow and command order.

Each command adds inclusions to the single global graph:

$$\begin{aligned}x &:= \&y &\Rightarrow \{ @y \} \subseteq \widehat{\mu}(@x), \\x &:= \text{null} &\Rightarrow \{ \varepsilon \} \subseteq \widehat{\mu}(@x), \\x &:= y &\Rightarrow \widehat{\mu}(@y) \subseteq \widehat{\mu}(@x), \\x &:= \text{alloc}^a() &\Rightarrow \{ a \} \subseteq \widehat{\mu}(@x) \quad \wedge \quad \{ \varepsilon \} \subseteq \widehat{\mu}(a), \\x &:= *p &\Rightarrow \forall \hat{\ell} \in \widehat{\mu}(@p), \quad \widehat{\mu}(\hat{\ell}) \subseteq \widehat{\mu}(@x), *p &:= q &\Rightarrow \forall \hat{\ell} \in \widehat{\mu}(@p), \quad \widehat{\mu}(@q) \subseteq \widehat{\mu}(\hat{\ell}).\end{aligned}$$

Commands inside branches and loops generate the same constraints as unconditional commands: control flow is ignored.

Example — constraints generated by a program

Program

```
p := &x;  
while * do {  
  if * then q := p  
  else q := &y;  
  p := *q  
}
```

Generated constraints

$$\mathcal{C} = \left\{ \begin{array}{l} \{ @x \} \subseteq \hat{\mu}(@p), \\ \hat{\mu}(@p) \subseteq \hat{\mu}(@q), \\ \{ @y \} \subseteq \hat{\mu}(@q), \\ \forall \hat{\ell} \in \hat{\mu}(@q), \quad \hat{\mu}(\hat{\ell}) \subseteq \hat{\mu}(@p) \end{array} \right\}.$$

The loop and the conditional add no control-flow constraints: each command contributes its inclusions once.

The constraint transformer for the example

For the constraints on the previous slide, one application adds all edges currently implied by $\mathcal{C}$:

$$F_{\mathcal{C}}(\hat{\mu}) = \hat{\mu} \left[\begin{array}{lcl} @p & \mapsto & \hat{\mu}(@p) \cup \{@x\} \cup \bigcup_{\hat{\ell} \in \hat{\mu}(@q)} \hat{\mu}(\hat{\ell}), \\ @q & \mapsto & \hat{\mu}(@q) \cup \hat{\mu}(@p) \cup \{@y\} \end{array} \right].$$

Every other abstract cell is unchanged. All right-hand sides are evaluated in the input $\hat{\mu}$. Starting from no edges, iterate

$$\hat{\mu}_0 = \perp, \quad \hat{\mu}_{n+1} = F_{\mathcal{C}}(\hat{\mu}_n).$$

- $F_{\mathcal{C}}$ is monotone: constraints only add edges.
- Since there are finitely many possible edges, Kleene iteration terminates at the least solution.

Kleene iteration for the example

Starting from $\hat{\mu}_0 = \perp$, apply F_C simultaneously to all cells:

n	$\hat{\mu}_n(@p)$	$\hat{\mu}_n(@q)$
0	$\emptyset$	$\emptyset$
1	$\{@x\}$	$\{@y\}$
2	$\{@x\}$	$\{@x, @y\}$
3	$\{@x\}$	$\{@x, @y\}$

All other cells remain empty.

- The first application adds the two constant edges.
- The second propagates $\hat{\mu}_1(@p) = \{@x\}$ into $\hat{\mu}_2(@q)$.
- The load adds nothing because $\hat{\mu}_2(@x) = \hat{\mu}_2(@y) = \emptyset$.

Hence $F_C(\hat{\mu}_2) = \hat{\mu}_2$: the iteration has reached the least solution.

Remark. Here, the flow-sensitive analysis would produce the same result.

Exercise — constraint propagation

Consider

$$p := \text{alloc}^a(); \quad q := p; \quad *q := \&x; \quad r := *q.$$

Generate the flow-insensitive constraints and compute their least solution, starting from $\hat{\mu}_0 = \perp$.

Solution

$$\mathcal{C} = \left\{ \begin{array}{ll} \{a\} \subseteq \hat{\mu}(@p), & \{\varepsilon\} \subseteq \hat{\mu}(a), \\ \hat{\mu}(@p) \subseteq \hat{\mu}(@q), & \\ \forall \hat{\ell} \in \hat{\mu}(@q), & \{@x\} \subseteq \hat{\mu}(\hat{\ell}), \\ \forall \hat{\ell} \in \hat{\mu}(@q), & \hat{\mu}(\hat{\ell}) \subseteq \hat{\mu}(@r) \end{array} \right\}.$$

Propagation follows

$$a \in \hat{\mu}(@p) \implies a \in \hat{\mu}(@q) \implies \{\varepsilon, @x\} \subseteq \hat{\mu}(a) \implies \{\varepsilon, @x\} \subseteq \hat{\mu}(@r).$$

Therefore the least solution is

$$\hat{\mu}(@p) = \hat{\mu}(@q) = \{a\}, \quad \hat{\mu}(a) = \hat{\mu}(@r) = \{\varepsilon, @x\}.$$

Flow-insensitive analysis: pros and cons

Advantages

- Simple finite inclusion-constraint solver.
- Fast and scalable for may-alias and call-graph construction.

Limitations

- Ignores order: no kills, hence more aliases and weak updates.

In practice. A flow-insensitive analysis over SSA names recovers much of the precision of flow sensitivity for variables, but not for the heap.

Takeaways

1. A points-to graph soundly over-approximates the targets of every variable and heap cell.
2. Allocation sites make the heap abstraction finite, but merge objects and force weak updates.
3. The same abstraction supports either a flow-sensitive interpreter or a scalable flow-insensitive constraint solver.